

C 程序设计基础 理论练习

判断题

1. 从文件的逻辑结构上看，C 语言把文件看作数据流，并将数据按顺序以一维方式组织存储。
2. C 语言源程序是文本文件，目标文件和可执行文件是二进制文件。
3. 对于缓冲文件系统，在进行文件操作时，系统自动为每一个文件分配一块文件内存缓冲区（内存单元）。
4. 文件指针指向文件缓冲区中文件数据的存取位置。
5. 结构指针就是指向结构类型变量的指针。
6. 假设结构指针 `p` 已定义并正确赋值，其指向的结构变量有一个成员是 `int` 型的 `num`，则语句 `*p.num = 100;` 是正确的。
7. 使用结构指针作为函数参数只要传递一个地址值，因此，能够提高参数传递的效率。
8. 结构数组是结构与数组的结合体，与普通数组的不同之处在于每个数组元素都是一个结构类型的数据，包括多个成员项。
9. 对于结构数组 `s`，既可以引用数组的元素 `s[i]`，也可以引用 `s[i]` 中的结构成员。
10. 结构是变量的集合，可以按照对基本数据类型的操作方法单独使用其成员变量。
11. 结构是变量的集合，可以按照对同类型变量的操作方法单独使用其成员变量。
12. 在定义嵌套的结构类型时，必须先定义成员的结构类型，再定义主结构类型。
13. 一个结构类型变量所占的内存空间是其各个成员所占内存空间之和。
14. 不同类型的结构变量之间也可以直接赋值。
15. 字符串常量在内存中的存放位置由系统自动安排。
16. 字符串常量实质上是一个指向该字符串首字符的指针常量。
17. 调用 `printf` 函数，`%s` 的格式输出字符串时，字符数组名、字符指针和字符串常量都可以作为输出参数。
18. 调用 `strcmp` 函数比较字符串大小时，通常较长的字符串会较大。
19. 数组的基地址是在内存中存储数组的起始位置，数组名本身就是一个地址即指针值。
20. 对于定义 `int a[10], *p=a;` 语句 `p=a+1;` 和 `a=a+1;` 都是合法的。
21. 两个任意类型的指针可以使用关系运算符比较大小。
22. 冒泡排序效率较高，因为它只需要约 $n^2/2$ 次比较。
23. 要通过函数调用来改变主调函数中某个变量的值，可以把指针作为函数的参数。
24. 只要将多个指针作为函数的参数，函数就一定会返回多个值。
25. 执行语句 `int *p = 1000;` 后，指针变量 `p` 指向地址为 `1000` 的变量。
26. 语句 `int *p, q, r;` 定义了 3 个指针变量。
27. 执行语句 `int *p;` 后，指针变量 `p` 只能指向 `int` 类型的变量。

28. 不同类型的指针变量是可以直接相互赋值的。

29. 语句 `int *p; *p = 50;` 执行时，不会有任何错误。

30. 字符 `'\0'` 的 ASCII 码值为 0。

31. 字符串常量就是用一对双引号括起来的字符序列，它有一个结束标志 `'\0'`。

32. `"a"` 和 `'a'` 是等价的。

33. 二维数组定义的一般形式如下，其中的类型名指定数组中每个元素的类型。

```
类型名 数组名[行长度][列长度];
```

34. 二维数组定义的一般形式如下，其中的类型名指定数组名的类型。

```
类型名 数组名[行长度][列长度];
```

35. 二维数组定义的一般形式如下，其中的行长度和列长度都是整型常量表达式。

```
类型名 数组名[行长度][列长度];
```

36. 以下定义了一个三维数组 `array_day`。

```
int array_day[10][10][10];
```

37. 以下定义了一个二维数组 `tab`，该数组可以存放 7 个整型数据。

```
int tab[3][4];
```

38. 引用二维数组的元素要指定两个下标，即行下标和列下标，一般形式如下。

```
数组名[列下标][行下标]
```

39. 二维数组定义的一般形式如下：

```
类型名 数组名[行长度][列长度];
```

二维数组元素引用的一般形式如下，

```
数组名[列下标][行下标]
```

在引用数组元素时，行下标的合理取值范围是 $[0, \text{行长度}-1]$ ，列下标的合理取值范围是 $[0, \text{列长度}-1]$ 。行下标和列下标都不能越界。

40. 对以下定义的二维数组 `table`，其行下标的取值范围是 $[0, 2]$ ，列下标的取值范围是 $[0, 1]$ ，数组元素分别是：`table[0][0]`、`table[0][1]`、`table[1][0]`、`table[1][1]`、`table[2][0]`、`table[2][1]`，可以表示一个 3 行 2 列的矩阵。

```
int table[3][2];
```

41. 二维数组的元素在内存中按行/列方式存放，即先存放第 0 行的元素，再存放第 1 行的元素……其中每一行的元素再按照列的顺序存放。

42. 对以下定义的二维数组 `table`，其数组元素在内存中的存放顺序是：`table[0][0]`、`table[1][0]`、`table[2][0]`、`table[0][1]`、`table[1][1]`、`table[2][1]`。

```
int table[3][2];
```

43. 将二维数组的行下标和列下标分别作为循环变量，通过二重循环，就可以遍历二维数组，即访问二维数组的所有元素。由于二维数组的元素在内存中按行优先方式存放，将行下标作为外循环的循环变量，列下标作为内循环的循环变量，可以提高程序的执行效率。
44. 对二维数组初始化时，经常采用如下分行赋初值的方式，即在定义数组时，把初值表 `k` 中的数据依次赋给第 `k` 行的元素。
45. 对二维数组初始化时，也可以采用如下顺序赋初值的方式，即在定义数组时，根据数组元素在内存中的存放顺序，把初值表中的数据依次赋给元素。

```
类型名 数组名[行长度][列长度] = {初值表};
```

46. 一维数组定义的一般形式如下，其中的类型名指定数组中每个元素的类型。

```
类型名 数组名[数组长度];
```

47. 一维数组定义的一般形式如下，其中的类型名指定数组变量的类型。

```
类型名 数组名[数组长度];
```

48. 一维数组定义的一般形式如下，其中的数组长度是一个整型常量表达式，给定数组的大小。

```
类型名 数组名[数组长度];
```

49. 数组定义中，数组名后是用方括号括起来的常量表达式，不能用圆括号。

50. 以下定义了一个一维数组 `str`，该数组可以存放 81 个字符型数据。

```
char str[81];
```

51. 同一个数组中的每个元素都具有相同的数据类型，有统一的标识符即数组名，用不同的序号即下标来区分数组中的各元素。

52. 在定义数组之后，根据数组中元素的类型及个数，在内存中分配一段连续存储单元用于存放数组中的各个元素。

53. 数组定义后，数组名表示该数组所分配连续内存空间中第一个单元的地址，即首地址。

54. 数组定义后，数组名的值是一个地址，可以被修改。

55. 数组定义后，只能引用单个的数组元素，而不能一次引用整个数组。

56. 一维数组定义的一般形式如下：

```
类型名 数组名[数组长度];
```

数组元素引用的一般形式如下：

```
数组名[下标]
```

在引用数组元素时，下标的合理取值范围是 $[0, \text{数组长度}-1]$ ，下标不能越界。

57. 变量初始化的含义，就是在定义变量时对变量赋值。

58. 一维数组初始化的一般形式如下，即在定义数组时，对数组元素赋初值。其中初值表中依次放着数组元素的初值。

类型名 数组名[数组长度] = {初值表};

59. 表达式 $!x$ 等价于 $x != 1$ 。

60. 运算符“+”不能作为单目运算符。

61. 如果运算符“*”和“/”都是右结合的，则表达式 $10 * 6 / 5$ 值是 10。

62. $s(s-a)(s-b)(s-c)$ 是合法的 C 语言表达式。

63. 表达式 $!!6$ 的值是 6。

64. 表达式 $\sim(\sim 2 < 1)$ 的值是 5。

65. 表达式 $!(x > 0 || y > 0)$ 等价于 $!(x > 0) \&\&!(y > 0)$ 。

66. 若表达式 $\text{sizeof}(\text{int})$ 的值为 4，则 int 类型数据可以表示的最大整数为 $2^{31}-1$ 。

67. 表达式 $(z=0, (x=2) || (z=1), z)$ 的值是 1。

68. C 语言的运算符只有单目运算符和双目运算符两种。

69. 在所有的关系运算符 ($>=$ 、 $>$ 、 $==$ 、 $!=$ 、 $<=$ 、 $<$) 中，优先级最低的运算符是“ $==$ 、 $!=$ ”。

70. C 的 double 类型数据可以精确表示任何实数。

71. 在 C 语言中，常量和变量都有数据类型。

72. 与 float 型数据相比， double 型数据的精度高，取值范围大。

73. 08 是正确的整型常量。

74. 表达式 0195 是一个八进制整数。

75. 全局变量只能定义在程序的最前面，即第一个函数的前面。

76. 全局变量与局部变量的作用范围相同，不允许它们同名。

77. 为了便于计算机存储管理，C 语言把保存所有变量的数据区，分成动态存储区和静态存储区，静态局部变量被存放在动态存储区。

78. 自动变量如果没有赋值，其值被自动赋为 0。

79. 静态局部变量如果没有赋值，其存储单元中将是随机值。

80. 在 C 语言的函数定义中，如果不需要返回结果，就可以省略 return 语句。

81. 在 C 语言的函数定义中，如果省略了 return 语句，函数就无法返回主调函数。

82. 函数的实参和形参都可以是变量、常量和表达式。

83. 按照 C 语言的规定，在参数传递过程中，既可以将实参的值传递给形参，也可以将形参的值传递给实参，这种参数传递是双向的。

84. 按照 C 语言的规定，实参和形参的命名不得重复。

85. 在 C 语言中，可以将主调函数放在被调函数的后面，省略函数的声明。

86. 在嵌套循环（多层循环）中，每一层循环都应该使用自己的循环变量，以免互相干扰。

87. 以下程序段中，break 语句只执行一次。

```
n = 9;
for(i = 1; i <= n; i++){
    for(j = 1; j <= n; j++){
        if(i != 1 && j > i){
            break;
        }
        printf( "%4d", i*j );
    }
    printf("\n");
}
```

88. 执行以下程序段，将出现死循环。

```
for(i = 11; i <= 20; i++){
    for(i = 1; i <= 10; i++){
        printf( "%4d", i );
    }
    printf("\n");
}
```

89. 在嵌套循环（多层循环）中，每一层循环中都不应该改变其他层使用的循环变量的值，以免互相干扰。

90. 以下程序段的功能是输出 1~100 之间每个整数的各位数字之和。

```
for(num = 1; num <= 100; num++){
    s = 0;
    do{
        s = s + num % 10;
        num = num / 10;
    }while(num != 0);
    printf("%d\n", s);
}
```

91. C 语言的三种循环不可以互相嵌套。

92. 在多层循环中，一个 break 语句只向外跳一层。

93. 以下两个程序段等价，其功能是计算 $s = 1 + 2 + \dots + 10$ 。

```
/* 程序段 A*/  
s = 0;  
i = 1;  
while(i <= 10){  
    s = s + i;  
    i++;  
}
```

```
/* 程序段 B */  
s = 0;  
i = 1;  
while(1){  
    if(i > 10){  
        break;  
    }  
    s = s + i;  
    i++;  
}
```

94. 以下程序段中，当 **i** 大于 **10** 或者 **s** 大于 **20** 时，循环结束。

```
s = 0;  
i = 1;  
while(i <= 10){  
    s = s + i;  
    if(s > 20){  
        break;  
    }  
    i++;  
}
```

95. 在循环中使用 break 语句或者 continue 语句，其作用是相同的。

96. 执行以下 while 语句，将出现死循环。

```
s = 0;  
i = 1;  
while(1){  
    if(i > 10){  
        continue;  
    }  
    s = s + i;  
    i++;  
}
```

97. 以下两个程序段等价，其功能是计算 $s=1+3+5+7+9$ 。

```

/* 程序段 A*/
s = 0;
i = 0;
while(i < 10){
    i++;
    if(i % 2 == 0){
        continue;
    }
    s = s + i;
}

```

```

/* 程序段 B */
s = 0;
i = 0;
while(i < 10){
    i++;
    if(i % 2 != 0){
        s = s + i;
    }
}

```

98. 以下两个程序段不等价，执行程序段 B 将陷入死循环。

```

/* 程序段 A*/
s = 0;
for(i = 1; i <= 10; i++) {
    if(i % 2 == 0){
        continue;
    }
    s = s + i;
}

```

```

/* 程序段 B */
s = 0;
i = 1;
while(i <= 10){
    if(i % 2 == 0){
        continue;
    }
    s = s + i;
    i++;
}

```

99. 在以下程序段中，若变量已正确赋值，当条件 `i <= limit` 不满足（即 `i > limit`）或者条件 `m % i == 0` 满足时，循环结束。

```
int i, limit, m;
for(i = 2; i <= limit; i++){
    if(m % i == 0){
        break;
    }
}
```

100. continue 不是结束本次循环，而是终止整个循环的执行。

101. continue 只能用于循环体中。

102. do - while 语句的一般形式如下，第一次进入循环时，首先执行循环体语句，然后再检查循环控制条件，即计算表达式，若值为“真”，继续循环，直到表达式的值为“假”，循环结束，执行 do - while 的下一条语句。

```
do{
    循环体语句
}while(表达式);
```

103. do - while 语句的一般形式如下，其中的循环体语句至少执行一次。

```
do{
    循环体语句
}while(表达式);
```

104. while 语句和 do - while 语句的一般形式分别如下，其主要区别是：while 语句先判断循环条件，只有条件满足才进入循环，如果一开始条件就不满足，则循环一次都不执行。而 do - while 语句先执行循环体，后判断循环条件，所以无论循环条件的值如何，至少会执行一次循环体。

```
while (表达式)
    循环体语句
```

```
do{
    循环体语句
}while(表达式);
```

105. do-while 循环的 while 后的分号可以省略。

106. do-while 循环至少要执行一次循环语句。

107. while 语句的一般形式如下，当表达式的值为“真”时，循环执行，直到表达式的值为“假”，循环中止并继续执行 while 的下一条语句。

```
while (表达式)
    循环体语句
```

108. while 语句的一般形式如下，其中的循环体语句只能是一条语句。


```
while (表达式)
    循环体语句
```

109. 若变量已正确定义，以下 while 循环结束时，i 的值为 11。

```
i = 1;
while (i <= 10){
    printf("%d\n", i);
}
```

110. 若变量已正确定义，执行以下 while 语句将陷入死循环。

```
i = 1;
while (i <= 10) ;
    i++;
```

111. 如果 for 的循环体语句中没有使用 continue 语句，则以下 for 语句和 while 语句等价。

```
for (表达式 1; 表达式 2; 表达式 3)
    for 的循环体语句
```

```
表达式 1;
while (表达式 2) {
    for 的循环体语句;
    表达式 3;
}
```

112. 若变量已正确定义，执行以下程序段，输入负数时，循环结束。

```
total = 0;
scanf ("%d", &score);
while(score >= 0){
    total = total + score;
    scanf ("%d", &score);
}
```

113. 若变量已正确定义，执行以下程序段，输入 0 或者负数时，循环结束。

```
total = 0;
scanf ("%d", &score);
while(score > 0){
    total = total + score;
    scanf ("%d", &score);
}
```

114. 执行以下程序段，将输出 10。

```
int i = 10;
while (i < 10){
    printf("%d\n", i);
}
```

115. 对于如下的 switch 语句（使用 break）的一般形式，其执行流程是：首先求解表达式，如果表达式的值与某个常量表达式的值相等，则执行该常量表达式后的相应语句段；如果表达式的值与任何一个常量表达式的值都不相等，则执行 default 后的语句段，最后执行 break 语句，跳出 switch 语句。

```
switch (表达式) {
    case 常量表达式 1: 语句段 1; break;
    case 常量表达式 2: 语句段 2; break;
    ...
    case 常量表达式 n: 语句段 n; break;
    default:          语句段 n+1; break;
}
```

116. 在 switch 语句中，必须使用 default。

117. 设变量已正确定义，以下是合法的 switch 语句。

```
switch(choice){
    case choice == 1: price = 3.0; break;
    case choice == 2: price = 2.5; break;
    case choice == 3: price = 4.0; break;
    case choice == 4: price = 3.5; break;
    default: price = 0.0; break;
}
```

118. 对于如下 switch 语句（不使用 break）的一般形式，其执行流程是：首先求解表达式，如果表达式的值与某个常量表达式的值相等，则执行该常量表达式后的所有语句段，如果表达式的值与任何一个常量表达式的值都不相等，则执行 default 后的所有语句段。

```
switch (表达式) {
    case 常量表达式 1: 语句段 1;
    case 常量表达式 2: 语句段 2;
    ...
    case 常量表达式 n: 语句段 n;
    default: 语句段 n+1;
}
```

119. 在 switch 语句中，每一个的 case 常量表达式的值可以相同。

120. case 语句后如没有 break，顺序向下执行。

121. 在 switch 语句中，可以根据需要使用或不使用 break 语句。

122. C 语言中的字符常量指单个字符，用一对单引号及其所括起的字符来表示。

123. C 语言中，字符型数据的值就是其在 ASCII 字符集中的次序值，即 ASCII 码。

124. C 语言中，每个字符型数据在 ASCII 字符集中都有一个惟一的次序值，即 ASCII 码。

125. C 语言中，数字字符 `'1'` 的值（ASCII 码）就是数字 `1`。

126. C 语言中，数字字符 `'0'`，`'1'`，`'2'`，…，`'9'` 的 ASCII 码按降序连续排列。

127. C 语言中，大写字母 `'A'`，`'B'`，`'C'`，…，`'Z'` 的 ASCII 码按升序连续排列。

128. C 语言中，小写字母 `'a'`，`'b'`，`'c'`，…，`'z'` 的 ASCII 码按升序连续排列。

129. C 语言中，大小写字母 `'A'`，`'B'`，`'C'`，…，`'Z'`，`'a'`，`'b'`，`'c'`，…，`'z'` 的 ASCII 码按升序连续排列。

130. C 语言中，大写字母 `'M'` 的 ASCII 码值比小写字母 `'m'` 的 ASCII 码值大。

131. C 语言中，小写字母 `'a'` 的 ASCII 码值是大写字母 `'Z'` 的 ASCII 码值加 1。

132. 以下 C 语言表达式的值为“真”。

```
'D' - 'A' == 'd' - 'a'
```

133. C 语言中不能用 `printf` 函数输出字符。

134. C 语言中不能用 `scanf` 函数输入字符。

135. 执行语句 `putchar('S')` 后，在屏幕上显示的输出结果是 `'S'`。

136. 设变量已正确定义，执行以下程序段，顺序输入三个字符 `'Q'`，则输出 `Q`。

```
ch = getchar();  
putchar(ch);
```

137. 设变量已正确定义，执行以下程序段，顺序输入三个字符 `'k'`，则输出 `k`。

```
scanf("%c%c%c", &ch1, &ch2, &ch3);  
printf("%c", ch2);
```

138. 为了检查以下 `else-if` 语句的三个分支是否正确，至少需要设计 63 组测试用例，即 `ch` 的取值至少有 63 组（52 个大小写英文字母、10 个数字字符、其他字符）。

```
if((ch >= 'a' && ch <= 'z') || (ch >= 'A' && ch <= 'Z'))  
    letter ++;  
else if(ch >= '0' && ch <= '9')  
    digit ++;  
else  
    other ++;
```

139. `if-else` 语句的一般形式如下，其中的语句 1、语句 2 只能是一条语句。

```
if (表达式)  
    语句 1  
else  
    语句 2
```

140. 为了检查以下 if-else 语句的两个分支是否正确，至少需要设计 2 组测试用例，即 `number` 的取值至少有两组（偶数和奇数）。

```
if(number % 2 == 0){
    printf("The number is even.\n");
}else{
    printf("The number is odd.\n");
}
```

141. 为了检查以下 else-if 语句的三个分支是否正确，至少需要设计 5 组测试用例，即 `x` 的取值至少有五组（小于 0 的数、0、大于 0 且小于 15 的数、15 和大于 15 的数）。

```
if (x < 0){
    y = 0;
}else if (x <= 15){
    y = 4 * x / 3;
}else{
    y = 2.5 * x - 10.5;
}
```

142. 省略 else 的 if 语句的一般形式如下，若表达式的值为“真”，则执行语句 1；否则，就什么也不做。

```
if (表达式)
    语句 1
```

143. 执行以下程序段后，`y` 的值为-1。

```
x = -1;
if (x < 0){
    y = -1;
}
y = 0;
```

144. 如果变量已经正确定义，则执行以下程序段后，`x` 的值不变。

```
x = 4;
if (x < 0){
    y = -1;
}else if (x = 0){
    y = 0;
}else{
    y = 1;
}
```

145. 为了检查以下省略 else 的 if 语句的分支是否正确，至少需要设计 3 组测试用例，即 `grade` 的取值至少有三组（小于 60、等于 60、大于 60）。

```
if(grade < 60){
    printf("Fail\n");
}
```

146. 为了检查以下嵌套的 if 语句的三个分支是否正确，至少需要设计 3 组测试用例，即 `yournumber` 的取值至少有 3 组（等于 `mynumber`、大于 `mynumber`、小于 `mynumber`）。

```
if(yournumber == mynumber){
    printf("Good Guess!\n");
}else if(yournumber > mynumber ){
    printf("Too big!\n");
}else{
    printf("Too small!\n");
}
```

147. 以下程序段的功能是：将变量 `a`、`b` 的最大值赋给 `max`。

```
max = a;
if ( max < b ){
    max = b;
}
```

148. 假设 `k` 是整型变量，计算表达式 `1/k` 后结果的数据类型是整型。

149. 调用数学库函数时，编译预处理命令为 `include math.h`。

150. 假设 `k` 是整型变量，计算表达式 `1.0/k` 后结果的数据类型是浮点型。

151. 调用输入输出库函数时，编译预处理命令为 `#include <stdio.h>`。

152. 如果变量已经正确定义，则表达式 `fahr ++` 与 `fahr + 1` 等价。

153. for 语句的一般形式如下，其中的表达式 1 只执行一次。

```
for (表达式 1; 表达式 2; 表达式 3)
    循环体语句
```

154. for 语句的一般形式如下，若表达式 2 的值为“假”，则结束循环。

```
for (表达式 1; 表达式 2; 表达式 3)
    循环体语句
```

155. for 语句的一般形式如下，若表达式 2 的值为“真”，则先执行循环体语句，再计算表达式 3，然后继续循环。

```
for (表达式 1; 表达式 2; 表达式 3)
    循环体语句
```

156. C 程序中，用一对大括号 `{}` 括起来的多条语句称为复合语句，复合语句在语法上被认为是一条语句。

157. 循环体如包括有一个以上的语句，则必须用一对大括号 `{}` 括起来，组成复合语句，复合语句在语法上被认为是一条语句。

158. 在 C 语言中，仅由一个分号 (;) 构成的语句称为空语句，它什么也不做。

159. 执行以下程序段，sum 的值是 55。

```
int i, sum;
for (i = 1; i <= 10; i++){
    sum = sum + i;
}
```

160. 以下程序段的功能是计算 20 的阶乘。

```
int i;
double product;
product = 0;
for (i = 1; i <= 20; i++){
    product = product * i;
}
```

161. 执行以下程序段，sum 的值是 1.5。

```
int i, sum;
sum = 0;
for (i = 1; i <= 2; i++){
    sum = sum + 1.0/i;
}
```

162. 执行以下程序段，sum 的值是 0.75。

```
int i;
double sum;
sum = 0;
for (i = 2; i <= 4; i = i + 2){
    sum = sum + 1/i;
}
```

163. == 是关系运算符，用于比较两个操作数是否相等；而 = 是赋值运算符，表示对变量赋值。

164. 如果变量已经正确定义，则执行以下程序段后，x 的值不变。

```
if (x == 10) {
    y = 1;
} else {
    y = 0;
}
```

165. 如果变量已经正确定义，则执行以下程序段后，x 的值不变。

```
if (x = 20) {  
    y = 1;  
} else {  
    y = 0;  
}
```

166. 为了检查以下 if-else 语句的两个分支是否正确，至少需要设计 2 组测试用例，即 x 的取值至少有两组（不等于 0 的数和 0）。

```
if (x != 0){  
    y = 1 / x;  
} else{  
    y = 0;  
}
```

167. 为了检查以下 if-else 语句的两个分支是否正确，至少需要设计 3 组测试用例，即 x 的取值至少有三组（小于 15 的数、15 和大于 15 的数）。

```
if (x <= 15){  
    y = 4 * x / 3;  
}else{  
    y = 2.5 * x - 10.5;  
}
```

168. 执行以下程序段，输入 10，输出 10.00。

```
double x;  
scanf("%d", &x);  
printf("%.2f", x);
```

169. 执行以下程序段，输入 20，输出 20.00。

```
double x;  
scanf("%f", &x);  
printf("%.2f", x);
```

170. 执行以下程序段，输入 30，输出 30.00。

```
double x;  
scanf("x=%lf", &x);  
printf("%.2f", x);
```

171. 执行以下程序段，输入 1000 3 0.025，输出 1000#3#0.025。

```
int money, year;  
double rate;  
scanf("%d %d %lf", &money, &year, &rate);  
printf("%d#%d#%.3f", money, year, rate);
```

172. 执行以下程序段，输入 `1001 3 0.025`，输出 `1001#3#0.025`。

```
int money, year;
double rate;
scanf("%d %lf %d ", &money, &year, &rate);
printf("%d#%d#%.3f", money, year, rate);
```

173. 执行以下程序段，输入 `1002 0.025 3`，输出 `1002#3#0.025`。

```
int money, year;
double rate;
scanf("%d%lf%d", &money, &rate, &year);
printf("%d#%d#%.3f", money, year, rate);
```

174. 执行以下程序段，输入 `1003 3 0.025`，输出 `1003#3#0.025`。

```
int money, year;
double rate;
scanf("%d %d %lf", &money, &year);
printf("%d#%d#%.3f", money, year, rate);
```

175. if-else 语句的一般形式如下，若表达式的值为“真”，则执行语句 1；否则，就执行语句 2。

```
if (表达式)
    语句 1
else
    语句 2
```

176. 在 C 程序运行过程中，其值不能被改变的量称为常量，其值可以改变的量称为变量。

177. 在 C 语言的数据类型中，`float` 的含义是单精度浮点型，`double` 的含义是双精度浮点型。

178. 以下程序段符合 C 语言语法。

```
k = 1;
int k;
```

179. C 程序中定义的变量，代表内存中的一个存储单元。

180. 在 C 程序中，`APH` 和 `aph` 代表不同的变量。

181. 在 C 语言中，单目运算符需要 2 个操作数。

182. 若变量定义为 `int fahr;`，则 `5(fahr-32)/9` 是符合 C 语言语法的表达式。

183. 若变量定义为 `double x;`，则 `x % 2` 是符合 C 语言语法的表达式。

184. 若变量定义为 `int n;`，当 `n` 的绝对值大于 1 时，则表达式 `1/n` 的值恒为 0。

185. 若变量定义为 `int fahr;`，则表达式 `5 * (fahr - 32) / 9` 和表达式 `5 / 9 * (fahr - 32)` 是等价的。

186. 若变量定义为 `int x, y;`，则 `x + y = 22` 是符合 C 语言语法的表达式。

187. 假设赋值运算符的优先级比算术运算符高，执行以下程序段后，`n` 的值为 10。


```
int n;  
n = 10 + 2;
```

188. 假设某段 C 语言程序中定义了三个变量 `a`、`b` 和 `c` 并且三个变量都不为 0，则表达式 `a / b * c` 和 `a * c / b` 是等价的，其值相同。

189. 执行以下程序段后，输出 `i = 10, j = 20`。

```
int i, j;  
i = 10;  
j = 20;  
printf("i = %d, j = %d", j, i);
```

190. C 语言的标识符由字母、数字和其他任意字符组成。

191. 在 C 语言中，标识符中的英文字母是区分大小写的。

192. 在 C 语言中，一个语句可以书写在不同行上。

193. C 语言的源程序是可以直接运行的。

194. 程序调试就是找出并改正 C 源程序中的语法错误。

195. 单步跟踪（Trace Step by Step）是常用的程序调试方法，即一步一步跟踪程序的执行过程，同时观察变量的变化情况。

选择题

1. 以下语句将输出

```
#include <stdio.h>
printf("%d %d %d", NULL, '\0', EOF);
```

- A. 0 0 1 B. 0 0 -1 C. NULL EOF D. 1 0 EOF

2. 定义 FILE *fp; 则文件指针 fp 指向的是

- A. 文件在磁盘上的读写位置 B. 文件在缓冲区上的读写位置
C. 整个磁盘文件 D. 文件类型结构体

3. 缓冲文件系统的文件缓冲区位于

- A. 磁盘缓冲区中 B. 磁盘文件中 C. 内存数据区中 D. 程序文件中

4. 直接使文件指针重新定位到文件读写的首地址的函数是

- A. ftell()函数 B. fseek()函数 C. rewind()函数 D. ferror()函数

5. 以下可作为函数 fopen 中第一个参数的正确格式是

- A. c:user\text.txt B. c:\user\text.txt C. "c:\user\text.txt" D. "c:\\user\\text.txt"

6. 函数 fgetc 的作用是从指定文件读入一个字符, 该文件的打开方式可以是

- A. 只写 B. 追加 C. 读或读写 D. 答案 B 和 C 都正确

7. 若以 “a+” 方式打开一个已存在的文件, 则以下叙述正确的是

- A. 文件打开时, 原有文件内容不被删除, 位置指针移到文件末尾, 可作添加和读操作
B. 文件打开时, 原有文件内容不被删除, 位置指针移到文件开头, 可作重写和读操作
C. 文件打开时, 原有文件内容被删除, 只可作写操作
D. 以上各种说法都不正确

8. 已知函数的调用形式 fread(buffer, size, count, fp); 其中 buffer 代表的是

- A. 一个整数变量, 代表要读入的数据项总数
B. 一个文件指针, 指向要读的文件
C. 一个指针, 指向要读入数据的存放地址
D. 一个存储区, 存放要读的数据项

9. 利用函数 fseek 可实现的操作是

- A. 改变文件指针 fp 的值 B. 文件的顺序读写
C. 文件的随机读写 D. 以上答案均正确

10. 若 fopen()函数打开文件失败, 其返回值是

- A. 1 B. -1 C. NULL D. ERROR

11. 若读文件还未读到文件末尾, feof()函数的返回值是

- A. -1 B. 0 C. 1 D. 非 0

12. fputc(ch,fp) 把一个字符 ch 写到 fp 所指示的磁盘文件中, 若写文件失败则函数的返回值为

- A.0 B.1 C.EOF D.非 0

13. 下面定义结构变量的语句中错误的是

- A. `struct student{ int num; char name[20]; } s;`
B. `struct { int num; char name[20]; } s;`
C. `struct student{ int num; char name[20]; }; struct student s;`
D. `struct student{ int num; char name[20]; }; student s;`

14. 如果有以下定义语句, 则输出结果为

```
struct {  
    int x, y;  
} s[2] = { { 1, 3 }, { 2, 7 } };  
printf("%d\n", s[0].y/s[1].x );
```

- A.0 B.1 C.2 D.3

15. 根据下面的定义, 能打印出字母 M 的语句是

```
struct person{  
    char name[10];  
    int age;  
} c[10] = { "John", 17, "Paul", 19, "Mary", 18, "Adam", 16 };
```

- A. `printf("%c", c[3].name);` B. `printf("%c", c[3].name[1]);`
C. `printf("%c", c[2].name[0]);` D. `printf("%c", c[2].name[1]);`

16. 设有如下定义, 则对 data 中的 a 成员的正确引用是

```
struct sk{ int a; float b; } data, *p = &data;
```

- A. `(*p).data.a` B. `(*p).a` C. `p->data.a` D. `p.data.a`

17. 对于以下结构定义, `(*p) -> str++` 中的 `++` 加在

```
struct { int len; char *str; } *p;
```

- A. 指针 str 上 B. 指针 p 上 C. str 指向的内容上 D. 语法错误

18. 当定义一个结构变量时, 系统分配给它的内存空间大小是

- A. 各成员所需内存量的总和 B. 结构中第一个成员所需内存量
C. 成员中占内存量最大者所需容量 D. 结构中最后一个成员所需内存量

19. 如果结构变量 s 中的生日是“1984 年 11 月 11 日”, 下列对其生日的正确赋值是

```

struct student
{
    int no;
    char name[20];
    char sex;
    struct{
        int year;
        int month;
        int day;
    }birth;
};
struct student s;

```

- A. `year = 1984; month = 11; day = 11;`
- B. `birth.year = 1984; birth.month = 11; birth.day = 11;`
- C. `s.year = 1984; s.month = 11; s.day = 11;`
- D. `s.birth.year = 1984; s.birth.month = 11; s.birth.day = 11;`

20. C 语言中结构类型变量在程序执行期间

- A. 所有成员一直驻留在内存中
- B. 只有一个成员驻留在内存中
- C. 部分成员驻留在内存中
- D. 没有成员驻留在内存中

21. 若有下列定义，则以下不合法的表达式是

```

struct student{
    int num;
    int age;
};
struct student stu[3] = {{101, 20}, {102, 19}, {103, 20}}, *p = stu;

```

- A. `(p++)->num`
- B. `p++`
- C. `(*p).num`
- D. `p = &stu.age`

22. 以下程序的输出结果是

```

struct stu{
    int x;
    int *y;
} *p;
int dt[4] = {10, 20, 30, 40};
struct stu a[4] = {50, &dt[0], 60, &dt[1], 70, &dt[2], 80, &dt[3]};

int main( )
{
    p=a;
    printf("%d,", ++p->x);
    printf("%d,", (++p)->x);
    printf("%d", ++(*p->y));

    return 0;
}

```

A. 10,20,20

B. 50,60,21

C. 51,60,21

D. 60,70,31

23. 设有如下定义，则错误的输入语句是

```
struct ss{
    char name[10];
    int age;
    char sex;
} std[3], *p = std;
```

A. `scanf("%d", &(*p).age);`

B. `scanf("%d", p -> &age);`

C. `scanf("%c", &std[0].sex);`

D. `scanf("%c", &(p->sex));`

24. 下列程序的输出结果是

```
struct stu{
    char num[10];
    float score[3];
};

int main( )
{
    struct stu s[3] = {{"20021",90,95,85},{ "20022",95,80,75},{ "20023",100,95,90}};
    struct stu *p = s;
    int i;
    float sum = 0;

    for(i=0; i<3; i++){
        sum = sum + p->score[i];
    }
    printf("%.2f", sum);

    return 0;
}
```

A. 260.00

B. 270.00

C. 280.00

D. 285.00

25. 以下程序段的输出结果为

```
struct {
    int x;
    int y;
} s[2] = { { 1, 3 }, { 2, 7 } };

printf("%d", s[0].y/s[1].x );
```

A. 0

B. 1

C. 2

D. 3

26. 对于以下定义，错误的 scanf 函数调用语句是

```
struct pupil{
    char name[20];
    int age;
    int sex;
}pup[5];
```

- A. `scanf("%s", &pup[0].name);` B. `scanf("%d", &pup[1].age);`
 C. `scanf("%d", &pup[2].sex);` D. `scanf("%s", pup[4].name);`

27. 若程序中有下面的说明和定义，则会发生的情况是

```
struct abc {
    int x;
    char y;
};
struct abc s1, s2;
```

- A. 编译出错 B. 程序将顺利编译、连接、执行
 C. 能顺利通过编译、连接，但不能执行 D. 能顺利通过编译，但连接出错

28. 对于以下定义，不正确的叙述是

```
struct ex {
    int x;
    float y;
    char z ;
} example;
```

- A. `struct` 是定义结构类型的关键字 B. `example` 是结构类型名
 C. `x`, `y`, `z` 都是结构成员名 D. `struct ex` 是结构类型名

29. 下列语句定义 x 为指向 int 类型变量 a 的指针，正确的是

- A. `int a, *x = a;` B. `int a, *x = &a;`
 C. `int *x = &a, a;` D. `int a, x = a;`

30. 以下选项中，对基本类型相同的指针变量不能进行运算的运算符是

- A. `+` B. `-` C. `=` D. `==`

31. 若有以下说明，且 $0 \leq i < 10$ ，则对数组元素的错误引用是

```
int a[] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}, *p = a, i;
```

- A. `*(a+i)` B. `a[p-a+i]` C. `p+i` D. `*(&a[i])`

32. 下列程序的输出结果是

```
int main(void)
{
    int a[10] = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 }, *p = a+3;
    printf("%d", *++p);
    return 0;
}
```

A.3

B.4

C.a[4]的地址

D.非法

33. 对于下列程序，正确的是

```
void f(int *p)
{
    *p = 5;
}
int main(void)
{
    int a, *p;

    a = 10;
    p = &a;
    f(p);
    printf("%d", (*p)++);

    return 0;
}
```

A.5

B.6

C.10

D.11

34. 变量的指针，其含义是指该变量的

A.值

B.地址

C.名

D.一个标志

35. 以下程序的运行结果是

```
#include <stdio.h>
void sub(int x, int y, int *z)
{
    *z = y-x;
}
int main()
{
    int a,b,c;

    sub(10, 5, &a);
    sub(7, a, &b);
    sub(a, b, &c);
    printf("%d,%d,%d\n", a, b, c);

    return 0;
}
```

A.5,2,3

B.-5,-12,-7

C.-5,-12,-17

D.5,-2,-7

36. 对于以下程序段，则叙述正确的是

```
char s[ ]="china";
char *p;
p = s;
```

A. `s` 和 `p` 完全相同

B. 数组 `s` 中的内容和指针变量 `p` 中的内容相等

C. 数组 `s` 的长度和 `p` 所指向的字符串长度相等

D. `*p` 与 `s[0]` 相等

37. 下面程序段的运行结果是

```
char a[]="language", *p;  
p = a;  
while( *p != 'u') { printf("%c", *p - 32); p++; }
```

A.LANGUAGE B.language C.LANG D.langUAGE

38. 以下程序的输出结果是

```
int main(void)  
{  
    char a[ ] = "programming", b[ ] = "language";  
    char *p1 = a, *p2 = b;  
    int i;  
  
    for(i = 0; i < 7; i++){  
        if( *(p1+i) == *(p2+i) ){  
            printf("%c", *(p1+i));  
        }  
    }  
  
    return 0;  
}
```

A.gm B.rg C.or D.ga

39. 以下程序的输出结果是

```
int fun(char s[ ])  
{  
    int n = 0;  
    while ( *s <= '9' && *s >= '0'){  
        n = 10 * n + *s - '0';  
        s++;  
    }  
    return(n);  
}  
  
int main( )  
{  
    char s[10]={'6', '1', '*', '4', '*', '9', '*', '0', '*'};  
    printf("%d\n", fun(s));  
    return 0;  
}
```

A.9 B.61490 C.61 D.5

40. 用冒泡排序对 4, 5, 6, 3, 2, 1 进行从小到大排序, 第三趟排序后的状态为

A.4 5 3 2 1 6 B.4 3 2 1 5 6 C.3 2 1 4 5 6 D.2 1 3 4 5 6

41. 下列程序的输出结果是


```
# include <stdio.h>
int main(void){
    int a[10] = {0,1,2,3,4,5,6,7,8,9}, *p = a+3;
    printf("%d", p[2]);
    return 0;
}
```

A.3

B.4

C.5

D.非法

42. 下面程序的输出结果是

```
# include <stdio.h>
void fun (int *x, int *y){
    printf("%d%d", *x, *y);
    *x = 3;
    *y = 4;
}
int main(void){
    int x=1, y=2;

    fun(&x, &y);
    printf("%d%d", x, y);

    return 0;
}
```

A.2134

B.1212

C.1234

D.2112

43. 下面程序的输出结果是

```
#include <stdio.h>

void fun (int *x, int y){
    printf("%d%d", *x, y);
    *x=3;
    y=4;
}

int main(void){
    int x = 1, y = 2;

    fun(&y, x);
    printf("%d%d", x, y);

    return 0;
}
```

A.1234

B.1221

C.2131

D.2113

44. 如果有定义: `int m, n = 5, *p = &m;`, 与 `m = n` 等价的语句是

A. `m = *p;`

B. `*p = *&n;`

C. `m = &n;`

D. `m = **p;`

45. 以下选项中, 对基本类型相同的指针变量不能进行运算的运算符是

A. +

B. -

C. =

D. ==

46. 假定 int 类型变量占用两个字节，其有定义：`int x[10]={0, 2, 4};` 则数组 x 在内存中所占字节数是

A. 3

B. 6

C. 10

D. 20

47. 以下能正确定义数组并正确赋初值的语句是

A. `int N=5, b[N][N];`

B. `int a[1][2]={{1}, {3}};`

C. `int c[2][ ]={{1, 2}, {3, 4}};`

D. `int d[3][2]={{1, 2}, {34}};`

48. 若有定义：`int a[2][3];`，以下选项中对数组元素正确引用的是

A. `a[2][0]`

B. `a[2][3]`

C. `a[0][3]`

D. `a[1>2][1]`

49. 设有数组定义：`char array [ ]="China";` 则数组 array 所占的空间为

A. 4 个字节

B. 5 个字节

C. 6 个字节

D. 7 个字节

50. 下述对 C 语言字符数组的描述中错误的是

A. 字符数组可以存放字符串

B. 字符数组中的字符串可以整体输入、输出

C. 可以在赋值语句中通过赋值运算符"="对字符数组整体赋值

D. 不可以用关系运算符对字符数组中的字符串进行比较

51. 有以下定义：`char x[ ]="abcdefg"; char y[ ]={'a', 'b', 'c', 'd', 'e', 'f', 'g'};`，则正确的叙述为

A. 数组 x 和数组 y 等价

B. 数组 x 和数组 y 的长度相同

C. 数组 x 的长度大于数组 y 的长度

D. 数组 x 的长度小于数组 y 的长度

52. 以下程序的输出结果是

```
int main(void)
{
    int m[ ][3] = { 1, 4, 7, 2, 5, 8, 3, 6, 9 };
    int i, j, k=2;

    for (i=0; i<3; i++)
        printf ("%d ", m[k][i]);

    return 0;
}
```

A. 4 5 6

B. 2 5 8

C. 3 6 9

D. 7 8 9

53. 以下程序的输出结果是

```
int main(void){
    int aa[4][4]={ {1, 2, 3, 4}, {5, 6, 7, 8}, {3, 9, 10, 2}, {4, 2, 9, 6} };
    int i, s=0;

    for(i=0; i<4; i++)
        s += aa[i][1];
    printf("%d\n", s);

    return 0;
}
```

A.11

B.19

C.13

D.20

54. 下列程序段的功能是：计算数组 `x` 中相邻两个元素的和，依次存放到 `a` 数组中，然后输出 `a` 数组。程序段中待填空的①和②的正确选项是

```
int i;
int a[9], x[10];

for(i = 0; i < 10; i++){
    scanf("%d", &x[i]);
}
for( ① ; i < 10; i++ ) { /* 此处待填空 ① */
    a[i-1] = x[i] + ② ; /* 此处待填空 ② */
}
for(i = 0; i < 9; i++){
    printf("%d ", a[i]);
}
printf("\n");
```

A.① `i = 1` ② `x[i+1]`

B.① `i = 0` ② `x[i-1]`

C.① `i = 1` ② `x[i-1]`

D.① `i = 0` ② `x[i+1]`

55. 假定 `char` 类型变量占用 1 个字节，且数组定义如下，则数组 `tab_str` 在内存中所占字节数是

```
char tab_str [10][81];
```

A.810

B.10

C.81

D.0

56. 假定 `double` 类型变量占用 8 个字节，且数组定义如下，则数组 `point` 在内存中所占字节数是

```
double point [10][10];
```

A.0

B.800

C.10

D.20

E.100

57. 如果变量定义如下，则正确的语句是

```
int i, j, tab[3][4];
```

A.tab[0][] = 0;

B.tab[][3] = 3;

C.tab = 100;

D.

```
for(i = 1; i <= 3; i++){
    for(j = 1; i <= 4; j++){
        scanf("%d", &a[i][j]);
    }
}
```

E.

```
for(i = 0; i < 3; i++){
    for(j = 0; j < 4; j++){
        printf("%4d", tab[i][j]);
    }
    printf("\n");
}
```

58. 假定 `char` 类型变量占用 1 个字节，且数组定义如下，则数组 `str` 在内存中所占字节数是

```
char str[81];
```

A.0

B.10

C.80

D.81

59. 假定 `double` 类型变量占用 8 个字节，且数组定义如下，则数组 `length` 在内存中所占字节数是

```
double length [10];
```

A.0

B.10

C.80

D.160

60. 执行下列程序段后，变量 `c` 的值是

```
int a, b, c;

a = 10, b = 20;
c = (a%2 == 0) ? a : b;
```

A.0

B.5

C.10

D.20

61. 运算符 () 的优先级最高。

A.[]

B.+=

C.? :

D.++

62. 执行下列程序段的输出结果是

```
double x = 2.5, y = 4.7;
int a = 7;
printf("%.1f", x + a%3 * (int)(x+y) % 2 / 4);
```

A.2.5

B.2.8

C.3.5

D.3.8

63. 执行下列程序段的输出结果是

```
int num = 1234, s = 0;

while( num != 0){
    s += num % 10;
    num /= 10;
}
printf("%d", s);
```

A.4321

B.1234

C.0

D.10

64. 设字符型变量 `x` 的值是 064，表达式 `~x ^ x << 2 & x` 的值是

A.0333

B.333

C.0x333

D.020

65. 设 a 为整型变量，不能正确表达数学关系： $10 < a < 15$ 的 C 语言表达式是

A. `10 < a < 15`

B. `a == 11 || a == 12 || a == 13 || a == 14`

C. `a > 10 && a < 15`

D. `!(a <= 10) && !(a >= 15)`

66. 设以下变量均为 `int` 类型，表达式的值不为 9 的是

A. `(x = y = 8, x+y, x+1)`

B. `(x = y = 8, x+y, y+1)`

C. `(x = 8, x+1, y = 8, x+y)`

D. `(y = 8, y+1, x = y, x+1)`

67. 计算变量 x (x 大于 1) 整数部分位数的表达式，可以写作

A. `(int)log10(x)`

B. `log10(x)`

C. `log10(x)+1`

D. `(int)log10(x)+1`

68. C 语言中，关系表达式和逻辑表达式的值是

A.0

B.1

C.0 或 1

D.'T' 或 'F'

69. 判断变量 x、y 中有且只有 1 个值为 0 的表达式为

A. `!(x*y)&& x+y`

B. `(x*y)&& x+y`

C. `x*y==0`

D. `x==0 && y!=0`

70. 若变量已经被正确定义，为表示“变量 x 和 y 都能被 3 整除”，应使用的 C 表达式是

A. `(x%3 != 0) || (y%3 != 0)`

B. `(x%3 != 0) && (y%3 != 0)`

C. `(x%3 == 0) || (y%3 == 0)`

D. `(x%3 == 0) && (y%3 == 0)`

71. 若 a 是 32 位 int 类型变量，判断其 32 个 2 进位中末两位均为 1 的表达式为

A. `(a&3)==3`

B. `(a&3)==11`

C. `(a&11)==3`

D. `(a&11)==11`

72. 为表示“a 和 b 都不等于 0”，应使用的 C 语言表达式是

A. `(a!=0) || (b!=0)`

B. `a || b`

C. `!(a=0) && (b!=0)`

D. `a && b`

73. 执行下面程序中的输出语句后，输出结果是

```
int a;  
printf("%d\n", (a=3*5, a*4, a+5));
```

A.65

B.20

C.15

D.10

74. 设整型变量 a=2，则执行下列语句后，浮点型变量 b 的值不为 0.5 的是

A. `b = 1.0/a;`

B. `b = (float) (1/a);`

C. `b = 1/(float)a;`

D. `b = 1/(a*1.0);`

75. 若 `int n; float f = 13.8;`，则执行 `n = (int)f % 3` 后，n 的值是

A.1

B.4

C.4.333333

D.4.6

76. 下面 () 表达式的值为 4。

- A. `11 / 3` B. `11.0 / 3` C. `(float)11/3` D. `(int)(11.0/3+0.5)`

77. 若有定义 `int x=3, y=2` 和 `float a=2.5, b=3.5`，则表达式：`(x+y)%2+(int)a/(int)b` 的值是

- A. 0 B. 2 C. 1.5 D. 1

78. 已知字符 'c' 的 ASCII 码为 99，语句 `printf ("%d,%c", 'c', 'c'+1);` 的输出为

- A. 99,c B. 99,100 C. 99,d D. 语句不合法

79. 阅读以下程序段，如果从键盘上输入 1234567<回车>，则程序的运行结果是

```
int i,j;
scanf ("%3d%2d",&i,&j);
printf("i = %d, j = %d\n",i,j);
```

- A. i = 123, j = 4567 B. i = 1234, j = 567
C. i = 1, j = 2 D. i = 123, j = 45

80. 阅读以下程序段，如果从键盘上输入 abc<回车>，则程序的运行结果是

```
char ch;

scanf ("%3c",&ch);
printf ("%c",ch);
```

- A. a B. b C. c D. 语法出错

81. 已知字母 A 的 ASCII 码为十进制的 65，下面程序段的输出是

```
char ch1,ch2;
ch1='A'+5-'3';
ch2='A'+6-'3';
printf ("%d,%c\n",ch1,ch2);
```

- A. 67,D B. B,C C. C,D D. 不确定的值

82. 下面合法的 C 语言字符常量是

- A. `'\t'` B. `"A"` C. `'xx'` D. `A`

83. 下面程序段的输出是

```
int x=023;
printf ("%d\n",--x);
```

- A. 17 B. 18 C. 23 D. 24

84. 下面的程序段输出是

```
int k=11;
printf ("k=%d,k=%o,k=%x\n",k,k,k);
```

- A. k=11,k=12,k=11 B. k=11,k=13,k=13

C. k=11,k=013;k=0xb

D. k=11,k=13,k=b

85. 下面的程序段输出是

```
short int a;  
int b = 65536;  
  
a = b;  
printf("%d\n", a);
```

A. 65536

B. 0

C. -1

D. 1

86. 在 C 语言程序中，若对函数类型未加显式说明，则函数的隐含类型为

A. void

B. double

C. char

D. int

87. 下列程序的输出结果是

```
fun(int a, int b, int c)  
{  
    c = a * b;  
}  
int main(void)  
{  
    int c;  
  
    fun(2, 3, c);  
    printf("%d\n", c);  
  
    return 0;  
}
```

A. 0

B. 1

C. 6

D. 无法确定

88. 建立自定义函数的目的之一是

A. 提高程序的执行效率

B. 提高程序的可读性

C. 减少程序的篇幅

D. 减少程序文件所占内存

89. 以下正确的函数定义形式是

A. double fun(int x, int y)

B. double fun(int x ; int y)

C. double fun(int x, int y);

D. double fun(int x, y)

90. 以下不正确的说法是

A. 实参可以是常量、变量或表达式

B. 实参可以是任何类型

C. 形参可以是常量、变量或表达式

D. 形参应与对应的实参类型一致

91. 以下正确的说法是

A. 实参与其对应的形参共同占用一个存储单元

B. 实参与其对应的形参各占用独立的存储单元

C. 只有当实参与其对应的形参同名时才占用一个共同的存储单元

D.形参是虚拟的，不占用内存单元

92.下列程序的输出结果是

```
# include <stdio.h>
int f(int n){
    static int k, s;
    n--;
    for(k=n; k>0; k--)
        s += k;
    return s;
}

int main(void){
    int k;
    k=f(3);
    printf("(%d,%d)", k, f(k));
    return 0;
}
```

A.(3,3)

B.(6,6)

C.(3,6)

D.(6,12)

93.函数f定义如下，执行语句“sum = f(5) + f(3);”后，sum的值应为

```
int f(int m)
{
    static int i=0;
    int s=0;

    for(;i<=m;i++)
        s+=i;

    return s;
}
```

A.21

B.16

C.15

D.8

94.有以下函数定义：

```
void fun(int n, double x){.....}
```

下列选项中的变量都已正确定义并赋值，则对函数fun的正确调用语句是

A. fun(int y, double m);

B. k=fun(10,12.5);

C. fun(x,n);

D. void fun(x,n);

95. 下列计算三角形面积函数的声明中，正确的是

A. double area(double a, double b, double c);

B. double area(int a, b, c);

C. int area(double a, double b, double c)

D. `double area(double a, b, c)`

96. C 语言中函数返回值的类型是由以下 () 决定的。

- A. 函数定义时指定的类型
- B. `return` 语句中的表达式类型
- C. 调用该函数时的实参的数据类型
- D. 形参的数据类型

97. 以下关于函数叙述中, 错误的是

- A. 函数未被调用时, 系统将不为形参分配内存单元
- B. 实参与形参的个数必须相等, 且实参与形参的类型必须对应一致
- C. 当形参是变量时, 实参可以是变量、常量或表达式
- D. 如函数调用时, 实参与形参都为变量, 则这两个变量不可能占用同一内存空间

98. C 语言中 `while` 和 `do-while` 循环的主要区别是

- A. `do-while` 的循环体至少无条件执行一次
- B. `while` 的循环控制条件比 `do-while` 的循环控制条件严格
- C. `do-while` 允许从外部转到循环体内
- D. `do-while` 的循环体不能是复合语句

99. 下列叙述中正确的是

- A. `break` 语句只能用于 `switch` 语句体中
- B. `continue` 语句的作用是使程序的执行流程跳出包含它的所有循环
- C. `break` 语句只能用在循环体内和 `switch` 语句体内
- D. 在循环体内使用 `break` 语句和 `continue` 语句的作用相同

100. 下列叙述中正确的是

- A. `do-while` 语句构成的循环不能用其他语句构成的循环来代替
- B. `do-while` 语句构成的循环只能用 `break` 语句退出
- C. 用 `do-while` 语句构成的循环, 在 `while` 后的表达式为非零时结束循环
- D. 用 `do-while` 语句构成的循环, 在 `while` 后的表达式为零时结束循环

101. 执行以下循环语句时, 下列说法正确的是

```
x = -1;
do {
    x = x * x;
} while (x == 0);
```

- A. 循环体将执行一次
- B. 循环体将执行两次
- C. 循环体将执行无限次
- D. 系统将提示有语法错误

102. 假设变量 `s`、`a`、`b`、`c` 均已定义为整型变量, 且 `a`、`c` 均已赋值 (`c` 大于 0), 则与以下程序段功能等价的

赋值语句是

```
s = a;
for(b = 1; b <= c; b++)
    s = s + 1;
```

- A. `s = a + b;` B. `s = a + c;` C. `s = s + c;` D. `s = b + c;`

103. 以下程序段的输出结果是

```
int main(void)
{
    int num = 0, s = 0;

    while(num <= 2){
        num ++;
        s += num;
    }
    printf("%d\n",s);

    return 0;
}
```

- A. 10 B. 6 C. 3 D. 1

104. 运行以下程序后，如果从键盘上输入 65 14<回车>，则输出结果为

```
int main(void)
{
    int m, n;

    printf("Enter m,n;");
    scanf("%d%d", &m,&n);
    while (m != n)
    {   while (m > n)        m = m - n;
        while (n > m)        n = n - m;
    }
    printf("m=%d\n",m);

    return 0;
}
```

- A. m=3 B. m=2 C. m=1 D. m=0

105. 下列程序段的输出结果是

```

int main(void)
{
    for(int i = 1; i < 6; i ++) {
        if( i % 2 != 0) {
            printf("#");
            continue;
        }
        printf("*");
    }
    printf("\n");

    return 0;
}

```

- A. #*#*#* B. ##### C. ***** D. *#*#*

106. 若变量已正确定义，对以下 while 循环结束条件的准确描述是

```

flag = 1;
denominator = 1;
item = 1.0;
pi = 0;
while(fabs(item) >= 0.0001){
    item = flag * 1.0 / denominator;
    pi = pi + item;
    flag = -flag;
    denominator = denominator + 2;
}

```

- A.item 的绝对值小于 0.0001 B.item 的绝对值大于 0.0001
C.item 的绝对值等于 0.0001 D.item 的绝对值不等于 0.0001

107. 若变量已正确定义，对以下 while 循环结束条件的准确描述是

```

flag = 1;
denominator = 1;
item = 1.0;
pi = 0;
while(fabs(item) > 0.0001){
    item = flag * 1.0 / denominator;
    pi = pi + item;
    flag = -flag;
    denominator = denominator + 2;
}

```

- A.item 的绝对值不大于 0.0001 B.item 的绝对值不小于 0.0001
C.item 的绝对值等于 0.0001 D.item 的绝对值不等于 0.0001

108. 若变量已正确定义，以下 while 循环正常结束时，累加到 pi 的最后一项 item 的值满足

```
flag = 1;
denominator = 1;
item = 1.0;
pi = 0;
while(fabs(item) >= 0.0001){
    item = flag * 1.0 / denominator;
    pi = pi + item;
    flag = -flag;
    denominator = denominator + 2;
}
```

- A.item 的绝对值小于 0.0001 B.item 的绝对值大于 0.0001
C.item 的绝对值大于等于 0.0001 D.item 的绝对值小于等于 0.0001

109. 若变量已正确定义，以下 while 循环正常结束时，累加到 pi 的最后一项 item 的值满足

```
flag = 1;
denominator = 1;
item = 1.0;
pi = 0;
while(fabs(item) >= 0.0001){
    pi = pi + item;
    flag = -flag;
    denominator = denominator + 2;
    item = flag * 1.0 / denominator;
}
```

- A.item 的绝对值小于 0.0001 B.item 的绝对值大于 0.0001
C.item 的绝对值大于等于 0.0001 D.item 的绝对值小于等于 0.0001

110. 能正确表示逻辑关系" $a \geq 10$ 或 $a \leq 0$ "的 C 语言表达式是

- A. `a>=10 or a<=0` B. `a>=0 | a<=10`
C. `a>=10 && a<=0` D. `a>=10 || a<=0`

111. 在嵌套使用 if 语句时，C 语言规定 else 总是

- A. 和之前与其具有相同缩进位置的 if 配对
B. 和之前与其最近的 if 配对
C. 和之前与其最近的且不带 else 的 if 配对
D. 和之前的第一个 if 配对

112. 下列叙述中正确的是

- A. break 语句只能用于 switch 语句
B. 在 switch 语句中必须使用 default

C. break 语句必须与 switch 语句中的 case 配对使用

D. 在 switch 语句中，不一定使用 break 语句

113. 有一函数 $y = \begin{cases} 1 & x > 0 \\ 0 & x = 0 \\ -1 & x < 0 \end{cases}$ ，以下程序段中错误的是

A.

```
if(x > 0) y = 1;
else if(x == 0) y = 0;
else y = -1;
```

B.

```
y = 0;
if(x > 0) y = 1;
else if(x < 0) y = -1;
```

C.

```
y = 0;
if(x >= 0);
if(x > 0) y = 1;
else y = -1;
```

D.

```
if(x >= 0)
if(x > 0) y = 1;
else y = 0;
else y = -1;
```

114. 下列程序段的输出结果是

```
int main(void)
{
    int a = 2, b = -1, c = 2;

    if(a < b)
        if(b < 0)
            c = 0;
    else c++;
    printf("%d\n", c);

    return 0;
}
```

A. 0

B. 1

C. 2

D. 3

115. 下列程序段的输出结果是

```
int main(void)
{
    int x = 1, a = 0, b = 0;

    switch(x)
    {
        case 0: b++;
        case 1: a++;
        case 2: a++; b++;
    }
    printf("a=%d,b=%d\n", a, b);

    return 0;
}
```

A. a=2,b=1

B. a=1,b=1

C. a=1,b=0

D. a=2,b=2

116. 在执行以下程序时，为使输出结果为：t=4，则给 a 和 b 输入的值应满足的条件是

```
int main(void)
{
    int a,b,s,t;

    scanf("%d,%d",&a,&b);
    s = 1; t = 1;
    if(a > 0) s = s + 1;
    if(a > b) t = s + t;
    else if(a == b) t = 5;
    else t = 2 * s;
    printf("t=%d\n",t);

    return 0;
}
```

A. a>b

B. a<b<0

C. 0<a<b

D. 0>a>b

117. 对于变量定义：`int a, b = 0;` 下列叙述中正确的是

A. `a` 的初始值是 `0`，`b` 的初始值不确定。

B. `a` 的初始值不确定，`b` 的初始值是 `0`。

C. `a` 和 `b` 的初始值都是 `0`。

D. `a` 和 `b` 的初始值都不确定。

118. 设变量已正确定义，以下（ ）与其它 switch 语句不等价。

A.

```
switch(choice){
    case 1: price = 3.0; break;
    case 2: price = 2.5; break;
    case 3: price = 4.0; break;
    case 4: price = 3.5; break;
    default: price = 0.0; break;
}
```

B.

```
switch(choice){
    default: price = 0.0; break;
    case 4: price = 3.5; break;
    case 3: price = 4.0; break;
    case 2: price = 2.5; break;
    case 1: price = 3.0; break;
}
```

C.

```
switch(choice){
    case 1: price = 3.0; break;
    case 2: price = 2.5; break;
    case 3: price = 4.0; break;
    case 4: price = 3.5; break;
}
```

D.

```
switch(choice){
    case 1: price = 3.0; break;
    case 2: price = 2.5; break;
    case 3: price = 4.0; break;
    case 4: price = 3.5; break;
}
```

119. 设变量已正确定义，以下（ ）与其它 switch 语句不等价。

A.

```
switch(op){
    case '+': printf("%d\n", value1 + value2); break;
    case '-': printf("%d\n", value1 - value2); break;
    case '*': printf("%d\n", value1 * value2); break;
    default: printf("Error\n"); break;
}
```

B.

```
switch(op){
    default: printf("Error\n"); break;
    case '*': printf("%d\n", value1 * value2); break;
    case '-': printf("%d\n", value1 - value2); break;
    case '+': printf("%d\n", value1 + value2); break;
}
```

C.

```
if(op != '+' && op != '-' && op != '*'){
    printf("Error\n");
}else{
    switch(op){
        case '+': printf("%d\n", value1 + value2); break;
        case '-': printf("%d\n", value1 - value2); break;
        case '*': printf("%d\n", value1 * value2); break;
    }
}
```

D.

```
switch(op){
    case '+': printf("%d\n", value1 + value2); break;
    case '-': printf("%d\n", value1 - value2); break;
    case '*': printf("%d\n", value1 * value2); break;
}
```

120. 设变量已正确定义，为了检查以下 else-if 语句的分支是否正确，至少需要设计（ ）组测试用例。

```
if(op == '+'){
    printf("=%.2f\n", value1 + value2);
}else if(op == '-'){
    printf("=%.2f\n", value1 - value2);
}else if(op == '*'){
    printf("=%.2f\n", value1 * value2);
}else if(op == '/'){
    if(value2 != 0){
        printf("=%.2f\n", value1 / value2);
    }else{
        printf("Divisor can not be 0!\n");
    }
}else{
    printf("Unknown operator!\n");
}
```

A.7

B.6

C.5

D.4

121. 改正下列程序中的_____处错误后，程序的运行结果是在屏幕上显示短句“Welcome to You!”。

```
#include <stdio.h>
int mian(void)
{
    printf(Welcome to You!\n");
    return 0;
}
```

- A.1 B.2 C.3 D.4

122. 若 fahr 为整型变量，则能正确表示以下数学式的 C 语言表达式是

$$\frac{5 \times (fahr - 32)}{9}$$

- A. 5*(fahr-32)/9 B. 5/9*(fahr-32) C. 5(fahr-32)/9 D. (fahr-32)/9*5

123. 下列 C 语言表达式的值不等于 1 的是

- A. 123/100 B. 901%10 C. 76%3 D. 625%5

124. 假设 i 和 j 是整型变量，以下语句_____的功能是在屏幕上显示形如 i * j = i*j 的一句乘法口诀。

例如，当 i=2, j=3 时，显示 2 * 3 = 6。

- A. printf("d * %d = %d\n", i, j, i*j);
 B. printf("%d * %d = %d\n", i, j, i*j);
 C. printf("%d * %d = %d\n", i, j);
 D. printf("%d = %d * %d\n", i, j, i*j);

125. 若 x 是 double 型变量，n 是 int 型变量，执行以下语句_____，并输入 3 1.25 后，x 的值是 1.25，n 的值是 3。

- A. scanf("%d%lf", &n, &x); B. scanf("%lf%d", &x, &n);
 C. scanf("%lf%d", &n, &x); D. scanf("%d, %lf", &n, &x);

126. 下列运算符中，优先级最低的是

- A. * B. = C. == D. %

127. 将以下 if-else 语句补充完整，正确的选项是

```
if(x >= y){
    printf("max = %d\n", x);
    _____
    printf("max = %d\n", y);
}
```

- A. else B. else{ C. }else{ D. }else E. else;

128. 为了检查以下 if-else 语句的两个分支是否正确，至少需要设计 3 组测试用例，其相应的输入数据和预期输出结果是

```
int x, y;
scanf("%d%d", &x, &y);
if(x >= y){
    printf("max = %d\n", x);
}else{
    printf("max = %d\n", y);
}
```


- A. 输入 3 和 4，输出 4；输入 5 和 100，输出 100；输入 4 和 3，输出 4。
- B. 输入 3 和 4，输出 4；输入 100 和 5，输出 100；输入 4 和 3，输出 4。
- C. 输入 3 和 4，输出 4；输入 5 和 5，输出 5；输入 -2 和 -1，输出 -1。
- D. 输入 3 和 4，输出 4；输入 5 和 5，输出 5；输入 4 和 3，输出 4。

129. 以下求 $n!$ 的函数可以正确计算 $21!$ ，正确的选项是

```

----- fact(int n){
    int i;
    ----- product;
    product = 1;
    for (i = 1; i <= n; i++){
        product = product * i;
    }
    return product;
}

```

- A. `double`
- B. `int`
- C. `float`
- D. `void`

130. 对 C 语言程序，以下说法正确的是

- A. `main` 函数是主函数，一定要写在最前面。
- B. 所有的自定义函数，都必须先声明。
- C. 程序总是从 `main` 函数开始执行的。
- D. 程序中只能调用库函数，不能自己定义函数。

131. 以下程序段_____的功能是计算序列 $1 + 1/2 + 1/3 + \dots$ 的前 N 项之和。

A.

```

int i, n, sum;
scanf("%d", &n);
sum = 0;
for (i = 1; i <= n; i++){
    sum = sum + 1.0/i;
}

```

B.

```

int i, n;
double sum;
scanf("%d", &n);
for (i = 1; i <= n; i++){
    sum = sum + 1.0/i;
}

```

C.

```

int i, n;
double sum;
scanf("%d", &n);
sum = 0;
for (i = 1; i <= n; i++){
    sum = sum + 1.0/i;
}

```

D.

```

int i, n;
double sum;
scanf("%d", &n);
sum = 0;
for (i = 1; i <= n; i++){
    sum = sum + 1/i;
}

```

E.

```
int i, n;
double sum;
scanf("%d", &n);
sum = 0;
for (i = 1, i <= n, i++){
    sum = sum + 1.0/i;
}
```

132. 以下程序段_____的功能是计算 n 的阶乘, 假设计算结果不超过双精度范围。

A.

```
int i, n;
double product;
scanf("%d", &n);
product = 0;
for (i = 1; i <= n; i++){
    product = product * i;
}
```

B.

```
int i, n, product;
scanf("%d", &n);
product = 1;
for (i = 1; i <= n; i++){
    product = product * i;
}
```

C.

```
int i, n;
double product;
scanf("%d", &n);
for (i = 1; i <= n; i++){
    product = product * i;
}
```

D.

```
int i, n;
double product;
scanf("%d", &n);
product = 1;
for (i = 1; i <= n; i++){
    product = product * i;
}
```

133. 以下说法中正确的是

- A. C 语言程序总是从第一个定义的函数开始执行
- B. 总是从 main() 函数开始执行
- C. C 语言程序中, 要调用的函数必须在 main() 中定义
- D. main() 函数必须放在程序的最开始部分

参考答案

判断题

1 - 5 TTTFT 6 - 10 FTTTT 11-15 TTTFT 16-20 TTFTF 21-25 FFTFF
26-30 FTFFT 31-35 TFTFT 36-40 TFFTT 41-45 TFTTT 46-50 TFTTT
51-55 TTTFT 56-60 TTTF 61-65 TFFTT 66-70 TFFTF 71-75 TTFFF
76-80 FFFFT 81-85 FFFFT 86-90 TFTTF 91-95 FTTTF 96-100 TTTTF
101-105 TTTTF 106-110 TTTFT 111-115 TTTFT 116-120 FFTFT
121-125 TTTTF 126-130 FTTF 131-135 FTFFF 136-140 FTTTT
141-145 TTFFT 146-150 TTTFT 151-155 TFTTT 156-160 TTTF
161-165 FFTTF 166-170 TTFFF 171-175 TFTFT 176-180 TTFTT
181-185 FFFTF 186-190 FTFFF 191-195 TTFFT

选择题

1-5 BBCCD 6-10 DACDC 11-15 BCDBC 16-20 BDADA
21-25 DCBBB 26-30 ABBBA 31-35 CBABB 36-40 DCDCC
41-45 CCDBA 46-50 DDDCC 51-55 CCBCA 56-60 BEDCC
61-65 AADAA 66-70 CDCAD 71-75 ADBBA 76-80 DDCDA
81-85 AABDB 86-90 DDBAC 91-95 BCCCA 96-100 ABACD
101-105 ABBCA 106-110 AAACD 111-115 CDCCA 116-120 CBCDB
121-125 CADBA 126-130 BCDAC 131 - 133 CDB